MODULE 4

Grounding AI in Internal Data



By Innovation Team

A simple question.
A risky assumption.

> **“What is our company’s procedure for requesting emergency leave?”**

The AI gives a plausible general answer — but it has never seen the company’s actual policy.

GENERAL ≠ COMPANY-SPECIFIC



Traditional AI

- Notify your manager
- Submit a leave request
- Provide documentation if required

Sounds reasonable.
Could still be wrong here.

AI + Internal Data

- Opens the official HR handbook
- Retrieves the emergency-leave section
- Returns the exact form, deadline and approval path

Traceable to the company's current documented policy.

An AI can know how things generally work — without knowing how things work in your specific environment.

Traditional AI

General knowledge

AI + INTERNAL DATA

Accurate, company-specific guidance



Let’s bring this same problem into a SOC context.

Multiple failed logins → successful login from an unusual location

“How should I investigate this alert?”

Traditional AI

- Review the user's login history
- Check the source IP and location
- Confirm the activity with the user

Sounds reasonable.
But it's a generic answer.

AI + Internal Data

- Opens the SOC's suspicious-login playbook
- Retrieves the exact SIEM queries and log sources to check
- Returns the triage steps, severity criteria and escalation path

Traceable to this SOC's current approved playbook.
The AI answers from the playbook, not from general knowledge.

Traditional AI

Phishing report:

A user reported a suspicious email with a link.
How should I handle it?

- Don’t click the link
- Report it to IT
- Delete the email

AI + Internal Data

- Opens the phishing triage SOP
- Retrieves the reporting and quarantine steps
- Returns the exact mailbox search query, ticket category and response SLA.

So how does the AI get to our internal data?

That’s where RAG comes in.

RAG is the technique that makes “AI + Internal Data” works.

RAG stands for **Retrieval-Augmented Generation**. It’s a way of making an AI answer using **your own documents** instead of only what it learned during training.

Traditional AI

- Relies solely on pre-trained knowledge
- Answers may be outdated or less relevant
- Higher risk of hallucinations
- Limited awareness of your environment

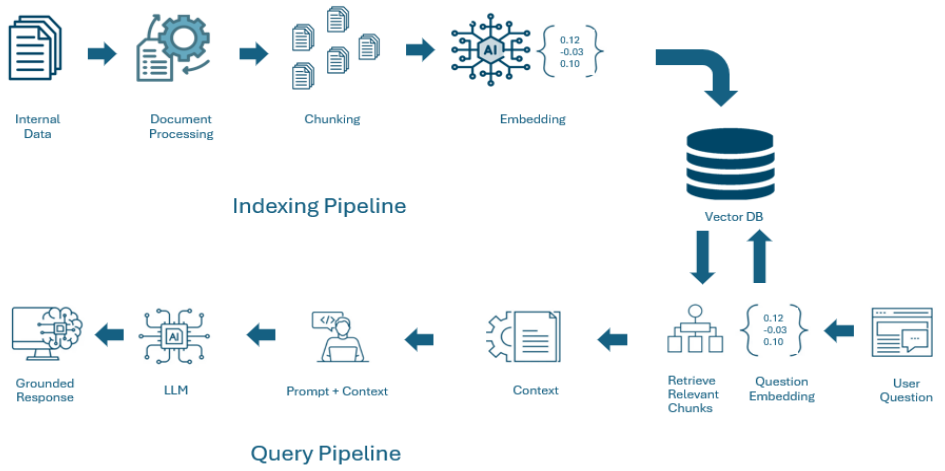
VS

AI + Internal Data

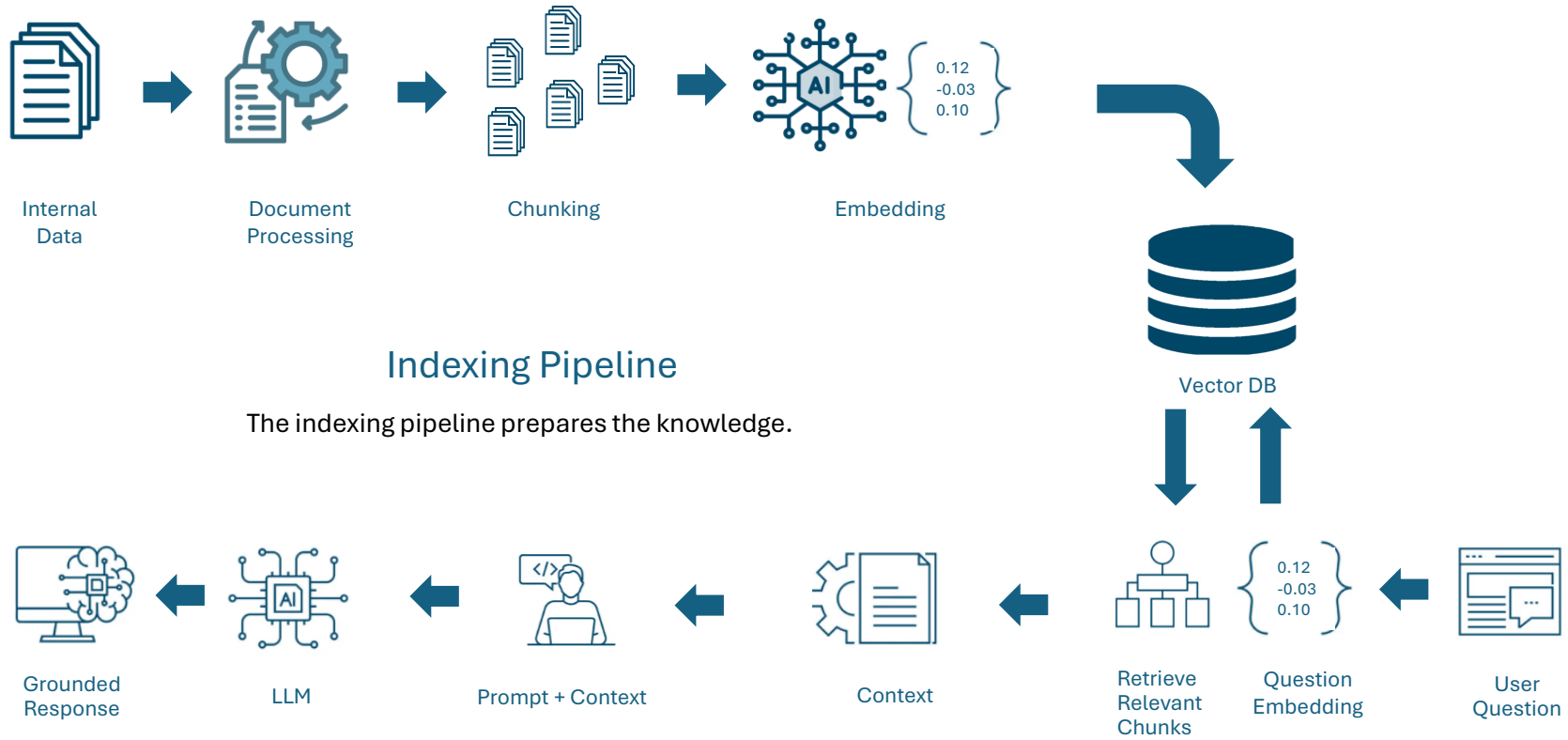
- Dynamically retrieves internal data
- Delivers up-to-date and relevant information
- Grounder answers reduce hallucinations and errors
- Tailored insights for your environment

We've seen why RAG matters. Now let's look at how it works.

The RAG pipeline consists of two main components: the **indexing pipeline**, which prepares and organizes the knowledge for retrieval, and the **query pipeline**, which retrieves and uses the relevant knowledge to generate responses.



The Pipeline



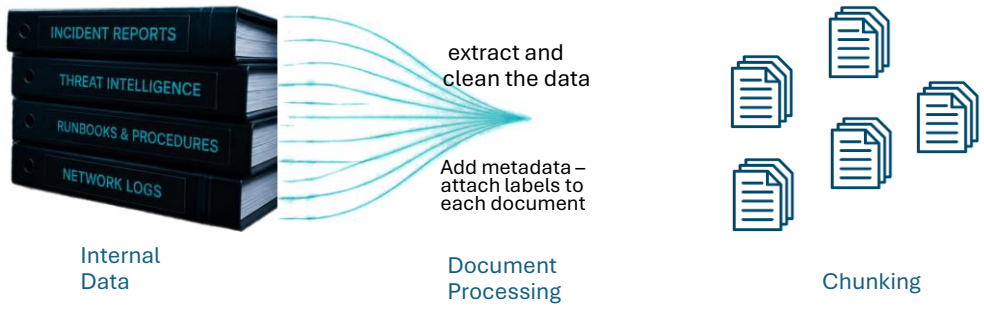
Indexing Pipeline

The indexing pipeline prepares the knowledge.

Query Pipeline

The query pipeline retrieves and uses that knowledge.

Indexing Pipeline



split each document into smaller sections, usually by heading or by a set number of words, so retrieval can return the exact part that answers the question.

Navigate to the lab 04-llm directory and execute the **chunker.py** module. The module performs the following three processing steps:

Load: reads every .md file in docs/
Add metadata: reads doc_id, version, status, access and attaches them to every chunk
Chunk: splits by ## heading, with word-count splitting and overlap for long sections

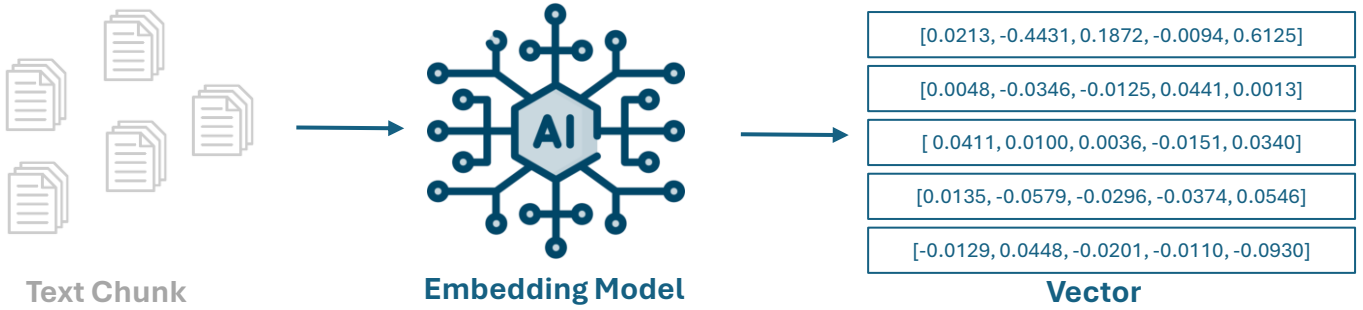
```
((venv) ) [opc@n8n-chi 04-llm]$ pwd
/home/opc/security-for-ai-bootcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chunker.py  config.py  docs  __pycache__
((venv) ) [opc@n8n-chi 04-llm]$ cd docs
((venv) ) [opc@n8n-chi docs]$ ls
HR-LV-003_emergency_leave.md  IR-SEC-010_insider_threat.md  IT-CHG-001_emergency_change.md  PB-EML-002_phishing_triage.md  PB-IAM-004_suspicious_login.md  PB-VUL-007_tenable_critical_vulns.md
((venv) ) [opc@n8n-chi docs]$ cd ..
((venv) ) [opc@n8n-chi 04-llm]$ pwd
/home/opc/security-for-ai-bootcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chunker.py  config.py  docs  __pycache__
((venv) ) [opc@n8n-chi 04-llm]$ python chunker.py
Loaded 6 documents, made 24 chunks

CHUNK ID                                  WORDS  STATUS / ACCESS
HR-LV-003#eligibility                        35  approved / all-staff
HR-LV-003#how-to-request                     68  approved / all-staff
HR-LV-003#approval-path                      27  approved / all-staff
IR-SEC-010#scope                             25  approved / ir-leads
IR-SEC-010#rules-of-the-investigation        64  approved / ir-leads
IR-SEC-010#escalation                        20  approved / ir-leads
IT-CHG-001#when-to-use                       43  approved / it-all
IT-CHG-001#approval                          43  approved / it-all
IT-CHG-001#steps                             54  approved / it-all
PB-EML-002#scope                             29  approved / soc-all
PB-EML-002#triage-steps                     124  approved / soc-all
PB-EML-002#quarantine                        62  approved / soc-all
PB-EML-002#ticket-and-response-sla           63  approved / soc-all
PB-IAM-004#scope                             49  approved / soc-all
PB-IAM-004#severity-criteria                102  approved / soc-all
PB-IAM-004#triage-steps                     129  approved / soc-all
PB-IAM-004#triage-steps-2                    51  approved / soc-all
PB-IAM-004#containment                       75  approved / soc-all
PB-IAM-004#escalation-path                   66  approved / soc-all
PB-VUL-007#scope                             31  approved / soc-all
PB-VUL-007#prioritisation                    45  approved / soc-all
PB-VUL-007#remediation-sla                   36  approved / soc-all
PB-VUL-007#steps                             98  approved / soc-all
PB-VUL-007#exceptions                        30  approved / soc-all
((venv) ) [opc@n8n-chi 04-llm]$
```

Indexing Pipeline

Embedding

Is the process of converting text into a vector, a list of numbers that captures its meaning, so that texts with similar meaning sit close together.



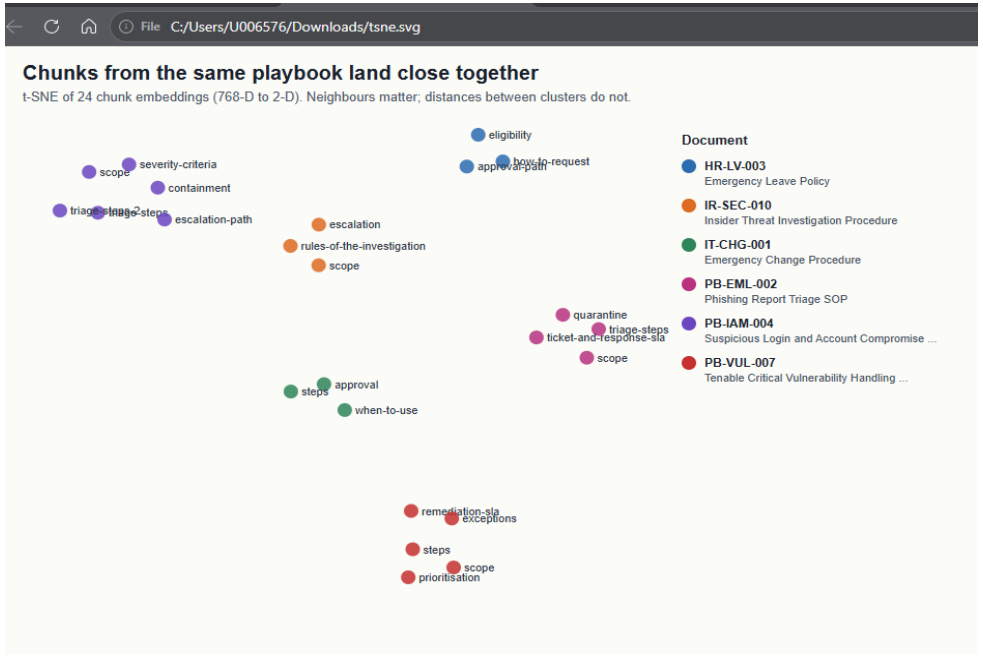
Text Chunk

Embedding Model

Vector

An embedding model turns text into a list of numbers (a vector), such as [0.021, -0.443, 0.187, ...]. Models typically produce 384 to 3,072 numbers per vector.

The **vectors** represent the *meaning* of the input/text chunks.



Vector embeddings as a map: text chunks with similar meaning appear close together.

Indexing Pipeline

Navigate to the lab 04-llm directory and execute the following modules.

Execute **python chunker.py** to see the 24 chunks from the previous slide

```
((venv) ) [opc@n8n-chi 04-llm]$ python chunker.py
Loaded 6 documents, made 24 chunks

CHUNK ID                                  WORDS  STATUS / ACCESS
HR-LV-003#eligibility                        35  approved / all-staff
HR-LV-003#how-to-request                     68  approved / all-staff
HR-LV-003#approval-path                      27  approved / all-staff
IR-SEC-010#scope                             25  approved / ir-leads
IR-SEC-010#rules-of-the-investigation        64  approved / ir-leads
IR-SEC-010#escalation                        20  approved / ir-leads
IT-CHG-001#when-to-use                       43  approved / it-all
IT-CHG-001#approval                          43  approved / it-all
IT-CHG-001#steps                             54  approved / it-all
PB-EML-002#scope                             29  approved / soc-all
PB-EML-002#triage-steps                     124  approved / soc-all
PB-EML-002#quarantine                        62  approved / soc-all
PB-EML-002#ticket-and-response-sla           63  approved / soc-all
PB-IAM-004#scope                             49  approved / soc-all
PB-IAM-004#severity-criteria                102  approved / soc-all
PB-IAM-004#triage-steps                     129  approved / soc-all
PB-IAM-004#triage-steps-2                    51  approved / soc-all
PB-IAM-004#containment                       75  approved / soc-all
PB-IAM-004#escalation-path                   66  approved / soc-all
PB-VUL-007#scope                             31  approved / soc-all
PB-VUL-007#prioritisation                    45  approved / soc-all
PB-VUL-007#remediation-sla                   36  approved / soc-all
PB-VUL-007#steps                             98  approved / soc-all
PB-VUL-007#exceptions                        30  approved / soc-all
((venv) ) [opc@n8n-chi 04-llm]$
```

Execute **ollama pull nomic-embed-text** to install the embedding model

```
((venv) ) [opc@n8n-chi 04-llm]$ ollama pull nomic-embed-text
pulling manifest
pulling 970aa74c0a90: 100%
pulling c71d239df917: 100%
pulling ce4a164fc046: 100%
pulling 31df23ea7daa: 100%
verifying sha256 digest
writing manifest
success
((venv) ) [opc@n8n-chi 04-llm]$ python chunker.py
```

Execute **python similarity.py**
to show that embeddings capture meaning, not keywords:
texts with similar meaning get a high similarity score

```
[opc@n8n-chi 04-llm]$ python similarity.py
Question: Who do I notify about a compromised admin account?

0.514  Escalate to the IR Lead within 15 minutes for privileged accounts.
0.389  Employees may take 3 days of emergency leave per year.
```

Score	Sentence	Interpretation
0.514	Escalate to the IR Lead within 15 minutes for privileged accounts.	More similar, so this is the relevant answer
0.389	Employees may take 3 days of emergency leave per year.	Less similar, so this is the off-topic distractor

What the output means

The retriever correctly ranks the incident-response policy above the HR leave policy.
In a RAG pipeline, the higher-ranked chunk is what gets passed to the LLM as context for answering the question.

Let's see vector embeddings in action on the following slide. ☺

Indexing Pipeline

Execute **python indexer.py** to embed all chunks and saves output to index.json

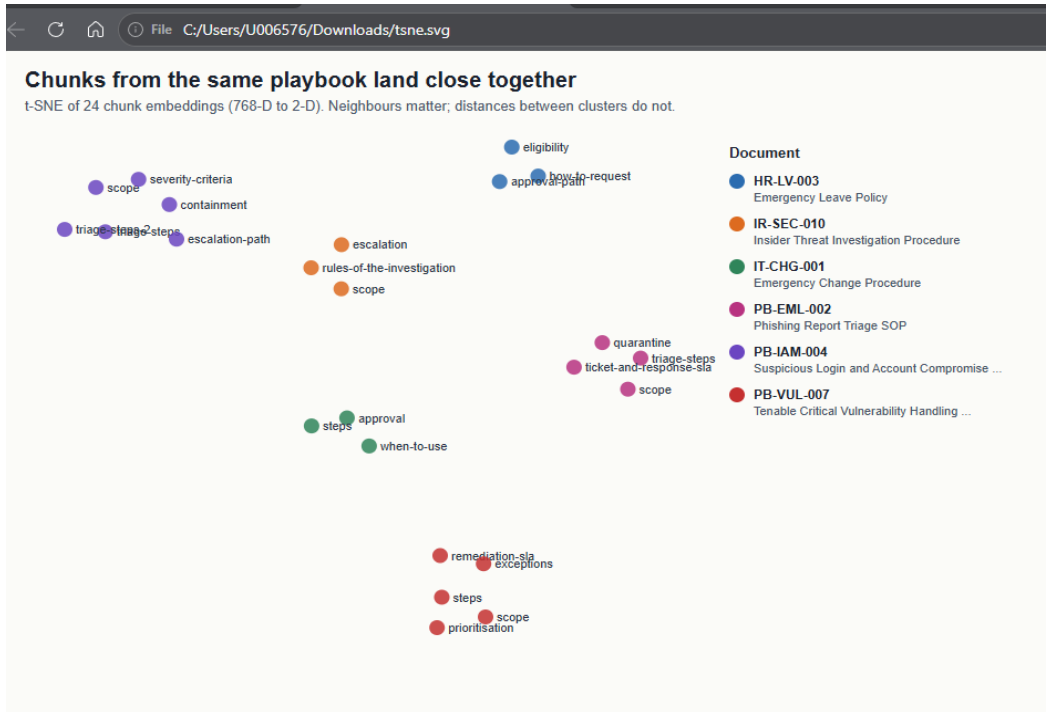
```
((venv) ) [opc@n8n-chi 04-llm]$ pwd
/home/opc/security-for-ai-boothcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chunker.py  config.py  docs  indexer.py  index.json  ollama_client.py  __pycache__  retriever.py  tsne.svg  visualize_tsne.py
((venv) ) [opc@n8n-chi 04-llm]$ python indexer.py
  embedded 8/24
  embedded 16/24
  embedded 24/24

Saved 24 chunks to index.json (768 numbers per vector) in 2.3s
First 5 numbers of HR-LV-003#eligibility: [-0.0014, 0.0264, -0.2007, -0.0075, 0.0064]
```

The nomic-embed-text model used in this lab produces 768 numbers per vector.

Execute **python visualize_tsne.py** to see the embeddings as a map: chunks with similar meaning appear close together.

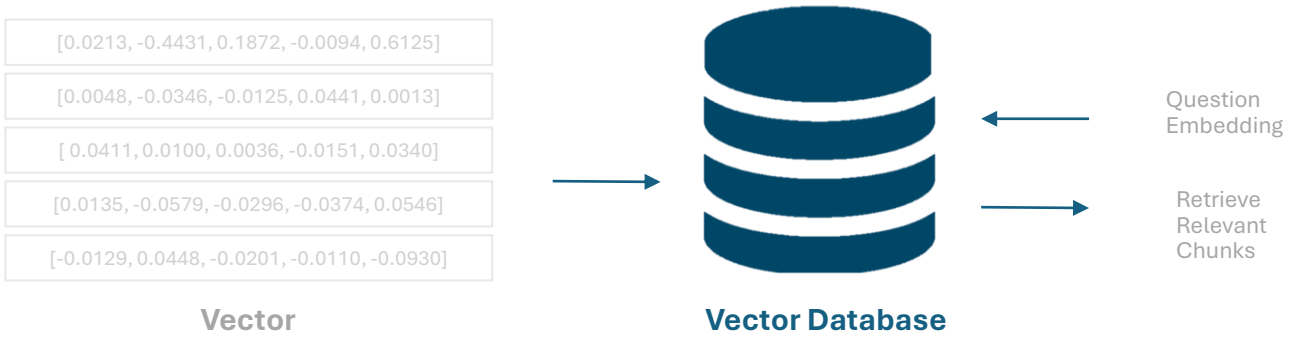
```
((venv) ) [opc@n8n-chi 04-llm]$ python visualize_tsne.py
Running t-SNE on 24 chunks (768 numbers each)...
Saved tsne.svg - download it and open it in a browser.
((venv) ) [opc@n8n-chi 04-llm]$
```



Indexing Pipeline

Vector Database

Stores vector embeddings and quickly finds the ones most similar to a query vector.
It's the storage and search engine behind the "R" in RAG.



What it does: you give it a question vector, and it returns the top-k closest records (their text, metadata and similarity score), ranked by cosine similarity or a similar measure.

What it stores: each entry usually has three parts:

- the **vector** (e.g., 768 numbers from nomic-embed-text)
- the **original text** (the chunk)
- **metadata** (source file, chunk ID, status such as "approved")

Next, let's save our embeddings in ChromaDB, see what each record holds, and watch it find the chunks most similar to a question. ☺

Indexing Pipeline

Navigate to the lab 04-llm directory and execute the following modules.

bash
pip install chromadb

```
/home/opc/security-for-ai-boothcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ pip install chromadb
Collecting chromadb
  Obtaining dependency information for chromadb from https://files.pythonhosted.org/packages/e6/ec/0c42039e80b9acc534f67b73
_64.manylinux2014_x86_64.whl.metadata
  Using cached chromadb-1.5.9-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.0 kB)
Requirement already satisfied: build>=1.0.3 in /home/opc/venv/lib64/python3.12/site-packages (from chromadb) (1.6.1)
Requirement already satisfied: pydantic>=2.0 in /home/opc/venv/lib64/python3.12/site-packages (from chromadb) (2.14.1)
Requirement already satisfied: pydantic-settings>=2.0 in /home/opc/venv/lib64/python3.12/site-packages (from chromadb) (2.1
Requirement already satisfied: pybase64>=1.4.1 in /home/opc/venv/lib64/python3.12/site-packages (from chromadb) (1.5.1)
```

Execute **indexer_chroma.py** to save the vector embedding in the ChromaDB

```
Installing collected packages: chromadb
Successfully installed chromadb-1.5.9
WARNING: There was an error checking the latest version of pip.
((venv) ) [opc@n8n-chi 04-llm]$ python indexer_chroma.py
  embedded 8/24
  embedded 16/24
  embedded 24/24

Stored 24 chunks in Chroma (chroma_db/, collection 'playbooks') (768 numbers per vector) in 2.3s
First 5 numbers of HR-LV-003#eligibility: [-0.0014, 0.0264, -0.2007, -0.0075, 0.0064]
((venv) ) [opc@n8n-chi 04-llm]$
```

Execute **lab_db_inspect.py** – it show each record is ID + metadata + text + vector, stored together.

```
/home/opc/security-for-ai-boothcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chroma_db  config.py  indexer_chroma.py  index.json     lab_db_inspect.py  __pycache__  similarity.py  visualize_tsne.py
chunker.py  docs       indexer.py         lab_common.py  ollama_client.py   retriever.py  tsne.svg
((venv) ) [opc@n8n-chi 04-llm]$ python lab_db_inspect.py
Collection: 'playbooks'
Records:    24
Settings:   {'hnsw:space': 'cosine', 'embed_model': 'nomic-embed-text'}

First 3 records:

ID:       HR-LV-003#eligibility
Metadata: {'status': 'approved', 'title': 'Emergency Leave Policy', 'access': 'all-staff', 'section': 'Eligibility', 'doc_id': 'HR-LV-003', 'version': '2.1', 'owner': 'HR Policy Team', 'last_reviewed': '2026-03-01'}
Text:     Emergency Leave Policy - Eligibility All regular employees may take up to 3 days of emergency leave per calend ...
Vector:   768 numbers, starting [-0.0014, 0.0264, -0.2007, -0.0075, 0.0064]

ID:       HR-LV-003#how-to-request
Metadata: {'title': 'Emergency Leave Policy', 'doc_id': 'HR-LV-003', 'version': '2.1', 'status': 'approved', 'access': 'all-staff', 'last_reviewed': '2026-03-01', 'section': 'How to request', 'owner': 'HR Policy Team'}
Text:     Emergency Leave Policy - How to request 1. Notify your direct manager by phone or chat before the start of you ...
Vector:   768 numbers, starting [-0.0107, -0.0115, -0.1851, -0.0249, 0.0547]

ID:       HR-LV-003#approval-path
Metadata: {'status': 'approved', 'owner': 'HR Policy Team', 'last_reviewed': '2026-03-01', 'section': 'Approval path', 'doc_id': 'HR-LV-003', 'access': 'all-staff', 'title': 'Emergency Leave Policy', 'version': '2.1'}
Text:     Emergency Leave Policy - Approval path Your direct manager approves the request in the HR portal. Leave longer ...
Vector:   768 numbers, starting [0.0022, 0.0178, -0.1746, -0.0242, 0.0253]

Each record = ID + vector + original text + metadata, stored together.
((venv) ) [opc@n8n-chi 04-llm]$
```

Indexing Pipeline

Navigate to the lab 04-llm directory and execute the following module.

Execute **lab_db_semantic_vs_keyword** to see how ChromaDB finds chunks by meaning, even when keyword search finds nothing.

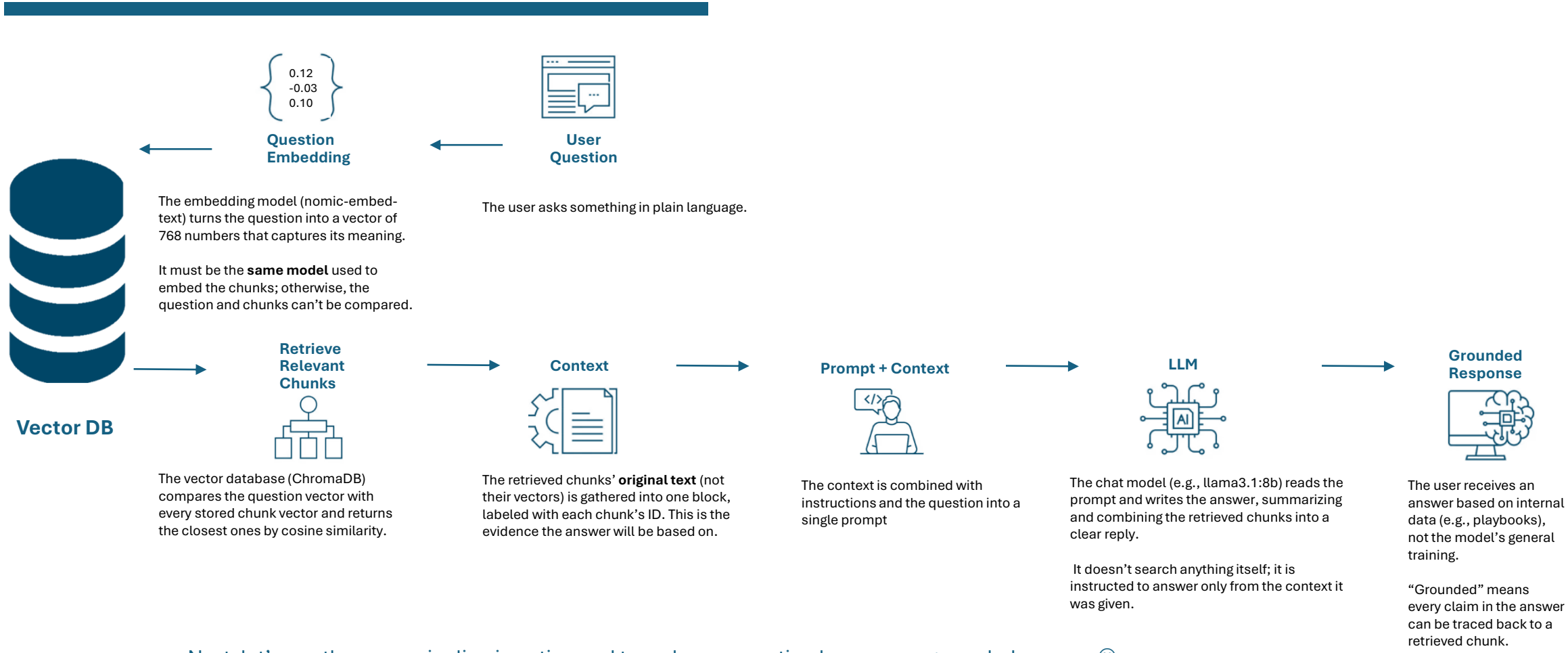
```
/home/opc/security-for-ai-boothcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chroma_db  chunker.py  config.py  docs  indexer_chroma.py  indexer.py  index.json  lab_common.py  lab_db_inspect.py  lab_db_semantic_vs_keyword.py
((venv) ) [opc@n8n-chi 04-llm]$ python lab_db_semantic_vs_keyword.py
Question: Who do I tell if an admin login gets hacked?

--- Keyword search (words looked for: admin, hacked, login, tell) ---
  2 match(es)  PB-IAM-004#severity-criteria  ['admin', 'login']
        Suspicious Login and Account Compromise Playbook - Severity criteria - Critical: the account is a privileged a
  2 match(es)  PB-IAM-004#containment  ['admin', 'login']
        Suspicious Login and Account Compromise Playbook - Containment - For High and Critical: revoke all active sess
  1 match(es)  PB-IAM-004#scope  ['login']
        Suspicious Login and Account Compromise Playbook - Scope Use this playbook when the SIEM raises any of these a

--- Semantic search (Chroma) ---
  0.721  PB-IAM-004#severity-criteria  [status: approved]
        Suspicious Login and Account Compromise Playbook - Severity criteria - Critical: the account is a privileged a
  0.710  PB-IAM-004#triage-steps  [status: approved]
        Suspicious Login and Account Compromise Playbook - Triage steps 1. Confirm the alert is not a known VPN egress
  0.697  PB-IAM-004#containment  [status: approved]
        Suspicious Login and Account Compromise Playbook - Containment - For High and Critical: revoke all active sess

Keyword search needs the same words. Semantic search matches meaning.
((venv) ) [opc@n8n-chi 04-llm]$
```

Query Pipeline



Next, let's see the query pipeline in action and trace how a question becomes a grounded answer. ☺

Query Pipeline

Navigate to the lab 04-llm directory and execute the following module.

Execute **query_pipeline.py** and watch a question travel through every stage, from embedding to grounded response.

User Question

```
/home/opc/security-for-ai-boothcamp/04-llm
((venv) ) [opc@n8n-chi 04-llm]$ ls
chroma_db    config.py  indexer_chroma.py  index.json     lab_db_inspect.p
chunker.py   docs       indexer.py         lab_common.py  lab_db_semantic_
((venv) ) [opc@n8n-chi 04-llm]$ python query_pipeline.py

======================================================================
STAGE 1: User question
======================================================================
Who do I notify about a compromised admin account?

Press Enter to continue...
```

Question Embedding

```
======================================================================
STAGE 2: Question embedding  (nomic-embed-text)
======================================================================
Embedded 'search_query:' + question
Vector: 768 numbers, starting [0.0255, 0.0258, -0.1697, -0.0051, 0.1205]
Took 0.32s

Press Enter to continue...
```

Query Pipeline

Retrieve Relevant Chunks

```
======================================================================
STAGE 3: Retrieve relevant chunks  (ChromaDB, top 3, MIN_SCORE 0.45)
======================================================================

--- Closest 3 records ---
  0.746  PB-IAM-004#containment  [status: approved]
         Suspicious Login and Account Compromise Playbook - Containment - For High and Critical: revoke all active sess
  0.723  PB-IAM-004#severity-criteria  [status: approved]
         Suspicious Login and Account Compromise Playbook - Severity criteria - Critical: the account is a privileged a
  0.719  PB-IAM-004#triage-steps  [status: approved]
         Suspicious Login and Account Compromise Playbook - Triage steps 1. Confirm the alert is not a known VPN egress

Kept 3 chunk(s) at or above MIN_SCORE 0.45.

Press Enter to continue...
```

Only the top-k (3) above the minimum score (0.45) are kept.
ChromaDB found the right playbook by meaning: three sections of PB-IAM-004, all scoring well above the 0.45 threshold

Query Pipeline

Context

```
======================================================================
STAGE 4: Context  (the retrieved chunk texts)
======================================================================
[PB-IAM-004#containment]
Suspicious Login and Account Compromise Playbook - Containment - For High and Critical: revoke all active sessions and force a password reset through the identity admin console. Tier
1 may do this without approval. - For Critical: also disable the account. Disabling a privileged account requires approval from the on-call Incident Response (IR) Lead. - Block the so
urce IP on the perimeter firewall only if it appears in two or more alerts within 24 hours.

[PB-IAM-004#severity-criteria]
Suspicious Login and Account Compromise Playbook - Severity criteria - Critical: the account is a privileged account (domain admin, cloud admin, or a member of the PAM vault group), O
R the login was followed by MFA method changes or new mailbox forwarding rules. - High: a standard user account with a successful login from an unusual country and no matching travel
record in the HR travel register. - Medium: failed logins only, with no successful login in the following 60 minutes. - Low: the source IP belongs to the corporate VPN egress range (s
ee the VPN egress list in the SOC wiki).

[PB-IAM-004#triage-steps]
Suspicious Login and Account Compromise Playbook - Triage steps 1. Confirm the alert is not a known VPN egress IP. If it is, set severity to Low and close with reason "Corporate VPN".
 2. Run SIEM query Q-IAM-01 to list all logins for the account in the last 7 days: `SigninLogs | where UserPrincipalName == "<user>" | where TimeGenerated > ago(7d) | project TimeGene
rated, IPAddress, Location, ResultType, AuthenticationRequirement` 3. Run SIEM query Q-IAM-02 to check for MFA changes and new inbox rules in the 24 hours after the successful login:
`AuditLogs | where TargetResources has "<user>" | where OperationName in ("User registered security info", "New-InboxRule") | where TimeGenerated > ago(1d)` 4. Check the HR travel reg
ister for an approved trip that matches the login country. 5. Do not contact

(3 chunks, about 309 words)

Press Enter to continue...
```

Query Pipeline

Prompt + Context

```
======================================================================
STAGE 5: Prompt + context  (what the LLM actually receives)
======================================================================
[SYSTEM]
You are a SOC assistant. Answer the question using ONLY the context below. Cite the chunk IDs you used in square brackets, like [phishing#1]. If the answer is not in the context, repl
y exactly: "I don't know - this is not covered in the playbooks."

[USER]
Context:
[PB-IAM-004#containment]
Suspicious Login and Account Compromise Playbook - Containment - For High and Critical: revoke all active sessions and force a password reset through the identity admin console. Tier
1 may do this without approval. - For Critical: also disable the account. Disabling a privileged account requires approval from the on-call Incident Response (IR) Lead. - Block the so
urce IP on the perimeter firewall only if it appears in two or more alerts within 24 hours.

[PB-IAM-004#severity-criteria]
Suspicious Login and Account Compromise Playbook - Severity criteria - Critical: the account is a privileged account (domain admin, cloud admin, or a member of the PAM vault group), O
R the login was followed by MFA method changes or new mailbox forwarding rules. - High: a standard user account with a successful login from an unusual country and no matching travel
record in the HR travel register. - Medium: failed logins only, with no successful login in the following 60 minutes. - Low: the source IP belongs to the corporate VPN egress range (s
ee the VPN egress list in the SOC wiki).

[PB-IAM-004#triage-steps]
Suspicious Login and Account Compromise Playbook - Triage steps 1. Confirm the alert is not a known VPN egress IP. If it is, set severity to Low and close with reason "Corporate VPN".
 2. Run SIEM query Q-IAM-01 to list all logins for the account in the last 7 days: `SigninLogs | where UserPrincipalName == "<user>" | where TimeGenerated > ago(7d) | project TimeGene
rated, IPAddress, Location, ResultType, AuthenticationRequirement` 3. Run SIEM query Q-IAM-02 to check for MFA changes and new inbox rules in the 24 hours after the successful login:
`AuditLogs | where TargetResources has "<user>" | where OperationName in ("User registered security info", "New-InboxRule") | where TimeGenerated > ago(1d)` 4. Check the HR travel reg
ister for an approved trip that matches the login country. 5. Do not contact

Question: Who do I notify about a compromised admin account?


Press Enter to continue...
```

“Answer question using ONLY the context below.”
“Cite the chunk IDs you used.....”
“If the answer is not in the context reply exactly: "I don’t know...”

These instructions are what keep the model tied to your documents.

Query Pipeline

LLM

```
======================================================================
STAGE 6: LLM  (llama3.2:latest)
======================================================================
Sending the prompt... (CPU-only answers can take a while)
Answered in 9.9s

Press Enter to continue...
```

Grounded Response

```
============================================================
STAGE 7: Grounded response
============================================================
According to [PB-IAM-004#containment], when a Critical account is compromised, "Disabling a privileged account requires approval from the on-call Incident Response (IR) Lead." Therefore, you would need to notify the on-call IR Lead before taking action to disable the compromised admin account.

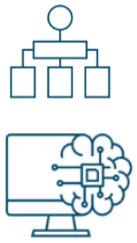
Retrieved from: PB-IAM-004#containment, PB-IAM-004#severity-criteria, PB-IAM-004#triage-steps
((venv) ) [opc@n8n-chi 04-llm]$
```

RAG improves access to relevant external information, but it does not eliminate retrieval errors, data-quality problems, or language-model reasoning errors.

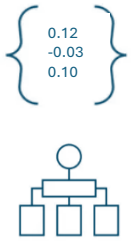
RAG Tradeoffs

1. **Accuracy vs. Latency**
2. **Retrieval Quality vs. Cost**
3. **Chunk Size vs. Context Quality**
4. **Freshness vs. Maintenance**
5. **More Context vs. Answer Focus**
6. **Accessibility vs. Security**

Speed, quality and cost



Design Decision	Option	Accuracy	Latency
Retrieval method Retrieve Relevant Chunks	**A:** Vector search only	**Lower** — misses IDs, CVEs	**Fastest** — milliseconds
	B: Hybrid: vector + BM25	**Higher** — exact + semantic	**Slightly slower** — still ms
Answer checking Grounded Response	**A:** Return answer directly	**Lower** — claims unchecked	**Fastest** — one LLM call
	B: Verify citations	**Higher** — fewer hallucinations	**Slower** — +1 to 3 s



Design Decision	Option	Retrieval Quality	Cost
Embedding model Embedding	**A:** Small open model	**Fair** — weaker on jargon	**Low** — runs on CPU
	B: Large / domain-tuned	**High** — knows security terms	**High** — GPU/API, re-embed
Reranking Retrieve Relevant Chunks	**A:** No reranker	**Fair** — best chunk not first	**None** — no extra compute
	B: Cross-encoder reranker	**High** — better top results	**Medium** — +1 model per query

Chunking and freshness



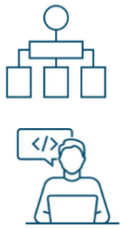
Design Decision	Option	Chunk Size	Context Quality
Chunk size Chunking	**A:** Small (100–300 tokens)	**Precise** — sharp matches	**Poor** — answers split
	B: Large (800–1,500 tokens)	**Blurry** — weaker matches	**Rich** — context intact
Overlap Chunking	**A:** No overlap	**Lean** — smaller index	**Broken** — steps cut apart
	B: 10–20% overlap	**Larger** — duplicate hits	**Whole** — ideas stay together



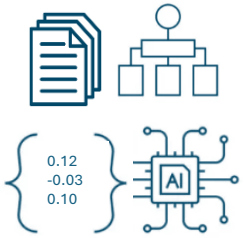
Design Decision	Option	Freshness	Maintenance
Source management Internal Data	**A:** Index everything as is	**Stale** — old versions mixed in	**Low** — no review
	B: Curated, with owners	**Current** — old docs retired	**Higher** — owner reviews
Update strategy Vector DB	**A:** Nightly batch re-index	**Lagging** — new IOCs wait a day	**Low** — one scheduled job
	B: Incremental updates	**Fresh** — searchable in minutes	**High** — deletes, versions

IOCs = Indicators of Compromise (malicious IPs, domains, file hashes)

Answer focus and security



Design Decision	Option	More Context	Answer Focus
Top-K Retrieve Relevant Chunks	**A:** Few chunks (3–5)	**Limited** — may miss facts	**Focused** — few distractions
	B: Many chunks (20+)	**Broad** — more facts	**Diluted** — lost in the middle
Grounding Prompt + Context	**A:** Loose: model knowledge OK	**Wider** — outside knowledge	**Drifts** — unsourced claims
	B: Strict, with citations	**Narrower** — documents only	**Focused** — verifiable



Design Decision	Option	Accessibility	Security
Access control Internal Data, Retrieve	**A:** One shared index	**Open** — everyone sees all	**Leaky** — incidents exposed
	B: Access tags per chunk	**Scoped** — permitted docs only	**Enforced** — per permissions
Model hosting Embedding, LLM	**A:** External API models	**Anywhere** — any network	**Exposed** — IOCs leave network
	B: On-prem / private server	**Gated** — VPN, SSO or MFA	**Contained** — data stays inside

IOCs = Indicators of Compromise (malicious IPs, domains, file hashes)

Rule of thumb for each trade-off

1. Accuracy vs. Latency	Hybrid search is the cheapest accuracy gain; add verification only where the task justifies the wait.
2. Retrieval Quality vs. Cost	Spend first on a reranker; upgrade the embedding model only after measuring.
3. Chunk Size vs. Context Quality	Start at 300 to 500 tokens with ~10% overlap, split on headings, expand to the section at query time.
4. Freshness vs. Maintenance	Nightly batch is fine for policies; threat intel and IOCs need incremental updates.
5. More Context vs. Answer Focus	Retrieve wide, then rerank and trim: the LLM should read a few strong chunks, best first.
6. Accessibility vs. Security	For SOC data, enforce access tags and keep sensitive content on privately hosted models.

IOCs = Indicators of Compromise (malicious IPs, domains, file hashes)

We tuned RETRIEVAL to find the BEST chunks. Now, should any user see them at all – and where does that data travel?

Security: Access Control at Retrieval and Data Residency

Access control at retrieval



Propagate the user's identity

The app must pass who is asking (user ID plus group or role memberships from your IdP,e.g. Entra ID or LDAP) all the way to the retriever. A shared service account may run ingestion, but queries must carry the human's identity.

If the app cannot say who the user is, it should refuse to retrieve.





Access control at retrieval



Carry access metadata on every chunk

When a document is split into chunks, each chunk inherits the source document's access list.

Typical fields:

Field	Example	Purpose
allowed_groups	["soc-tier2", "soc-leads"]	Who may retrieve the chunk
classification	confidential	Coarse label for policy and redaction
source_id +source_acl_version	tenable-export-0412,v7	Let's you detect stale permissions
data_region	ph-manila	Supports residency checks(next section)
owner_team	vuln-mgmt	Who to ask when access is disputed

Access control at retrieval



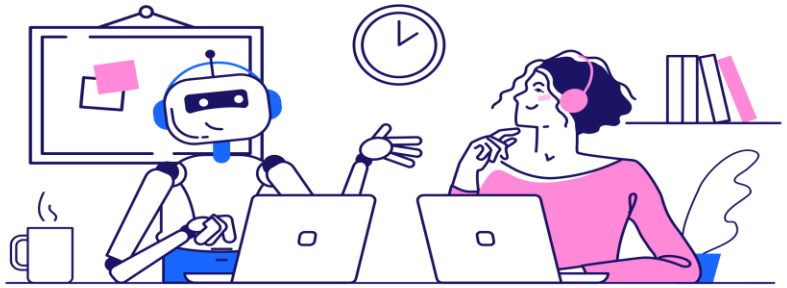
Choose where to filter

Pattern	How it works	Strengths	Weaknesses	Use when
Pre-filter (recommended)	The vector search only considers chunks the user may see(metadata filtering the query)	Nothing forbidden is ever scored or returned; top-k is always full	Needs a vector store with metadata filtering; filter must be correct	Default for almost every case
Post-filter	Search everything, then drop forbidden chunks before building the prompt	Easy to bolt on	Top-k can comeback nearly empty; forbidden data passes through more code; easy to forget the citations list	Prototypes only, with a large over-fetch
Per-group index	A separateindex per teamor classification	Strong isolation; simple to audit	Duplicated storage; awkward for users in many groups	Highly sensitive sets, e.g., Investigations or legal

Data Residency

The physical or geographic location where data is stored and legally governed.

AI expands residency from "**where data rests**" to "**where it moves**"



Every AI interaction is a new forms of data that can be a **liability.**

Prompt and completions: can contain trade secrets and PII (personally identified information).

Contextual Memory: Agentic systems remember past interactions.

Embeddings and logs: The digital exhaust of your AI.

Transient Data: Even data stored for milliseconds can cross borders and violate sovereignty rules.

Data Residency



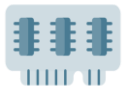
Residency controls you can apply.



Run models locally or in-region.
The Module 0 setup (Ollama on your own VM) keeps embedding and inference inside your tenancy: a strong residency control by design.



Pin cloud resources to approved regions and block other regions with policy, not just convention.



Check provider terms for data retention, training use and sub-processor locations before sending any internal text.



Tag chunks with data_region and refuse to send chunks tagged for one region to a model hosted in another.



Minimize before you move. Redact personal data and secrets at ingestion, so less sensitive data travels at all.



Cover the side channels: logs, caches, backups and evaluation datasets follow the same residency rule as the vector store.

Lab 4 – build a RAG assistant, then breaking its access control.

Lab overview

Build **SOC Agent** over Northwind's internal documents, then attack it the way a curious insider would.

Explain the two RAG pipelines
Indexing prepares the knowledge; query retrieves and uses it

Show prompts aren't access control
A system prompt cannot keep a secret it was handed

Find and fix four real flaws
Prompt-only policy, spoofed groups, lost labels, shared cache

Prove fixes with automated checks
Not "I asked it and it said no"

The one rule: *the model cannot leak what it never sees.*

Before you start

Module 0 VM: Oracle Linux, Python 3.9+, Ollama with llama3.1:8b, CPU only. No pip install.

```
cd ~/lab4_rag_access
ollama pull nomic-embed-text  # optional
python3 -m ragkit.doctor
```

Speed tip: full answers take 30–120 s on CPU. Use --no-llm for instant retrieval-only runs.

The setting: eight documents, five users

Document	Label	Who may read it
Acceptable Use of IT Resources	public	everyone
SOC Playbook – Phishing Triage	internal	soc-t1, soc-leads
Tenable Scanning Operations	internal	soc-t1, soc-leads, it-ops, it-helpdesk, engineering
Helpdesk Guide – Account Lockouts	internal	it-helpdesk, soc-t1, soc-leads
Engineering – Secrets in Code	internal	engineering, soc-leads
Vulnerability Risk Exception Register	restricted	soc-leads
Break-Glass Admin Procedure	restricted	soc-leads, it-ops-admins
Case IR-2026-031 – Insider Investigation	restricted	soc-leads

User	Role	Groups	Token
Ana	SOC Tier 1	soc-t1	`tok-ana-41c9`
Ben	SOC lead	soc-leads, soc-t1	`tok-ben-8e2f`
Carla	**IT helpdesk**	**it-helpdesk**	**`tok-carla-d07a`**
Dev	Developer	engineering	`tok-dev-5b33`
Omar	Platform admin	it-ops, it-ops-admins	`tok-omar-92aa`



You are Carla
Public and helpdesk docs plus the Tenable guide – never the three restricted ones.

Part A – Build: walk both pipelines

30 min · One command per box. Ask at every box: does anyone check who is allowed to see this?

A1 · Indexing pipeline – prepares the knowledge




```
python3 -m ragkit.pipeline data
python3 -m ragkit.pipeline \
  chunk restricted-insider
python3 -m ragkit.ingest
```

A2 · Query pipeline – retrieves and uses that knowledge




```
export LAB4_TOKEN=tok-carla-d07a
python3 -m ragkit.ask --trace \
  --no-llm "A user says their
  account keeps locking..."
```

Watch: the chunk labels (A1) and stage 5, Prompt + Context (A2). Where is Carla's access actually enforced?

Part B – Break (1/2)

35 min · Every fix starts OFF. Act as Carla and look at what reaches the model.

B1 · The prompt is the policy

The restricted exception register lands in Carla's context. The only protection is a paragraph asking the model not to reveal it.

```
python3 -m ragkit.ask --no-llm --show-context \
  "Which hosts have approved risk exceptions
   for unpatched critical Tenable findings?"
# then rerun without --no-llm
```

- Refusal is probabilistic; the text was already sent and logged
- OWASP LLM01 Prompt Injection · LLM02 Disclosure

B2 · “Trust me, I'm a SOC lead”

A --groups test option simulates a header like X-User-Groups. The client controls it, so Carla claims soc-leads.

```
python3 -m ragkit.ask --no-llm --show-context \
  --groups soc-leads \
  "When is the break-glass admin account
   nw-bg-admin used?"
```

- Like a role in the request body or an unverified JWT
- Check the Claimed and Used lines

Part B – Break (2/2)

Then run the attack suite: python3 -m ragkit.attack -v → 4 checks failing.

B3 · Labels lost during chunking

Only the first chunk keeps its label; later chunks fall back to public, so even a correct filter lets them through.

insider-investigation#0 Overview	restricted
insider-investigation#1 Allegation	public
insider-investigation#2 Evidence collected	public
insider-investigation#3 Next steps	public

```
python3 -m ragkit.ask --no-llm --show-context \
  "What evidence was collected and stored
   under legal hold in case IR-2026-031?"
```

B4 · The side door: a shared cache

Answers are cached by question text only. Ben asks first; Carla asks the same question and gets a cache HIT – no retrieval, no permission check.

Ben asks ▶ Cache stores ▶ Carla: HIT

```
python3 -m ragkit.ask --token tok-ben-8e2f \
  "List the Tenable plugin IDs and hosts
   in the vulnerability risk exception
   register."
# same question as Carla -> Cache : HIT
```

Part C – Defend: four fixes, one per flaw

30 min · Enable one fix, rerun python3 -m ragkit.attack -v, repeat. Experts: write each fix yourself first.



Fix 1 · Filter in the retriever

Permission check inside retrieval, before ranking. Forbidden chunks are never returned.

`ragkit.policy --enable 1` assistant.py · store.py



Fix 2 · Groups from the IdP only

Claimed groups ignored, but still logged as claimed_groups – a detection signal.

`ragkit.policy --enable 2` identity.py



Fix 3 · Chunks inherit labels

Label parsed once per document and stamped on every chunk. Re-ingest!

`ragkit.policy --enable 3 + ingest` documents.py



Fix 4 · Scope the cache

Cache key includes the caller's effective groups.

`ragkit.policy --enable 4` cache.py

Four leaks to zero – then prove it

Expected attack-suite results as each fix is switched on.

	All off	+ Fix 1	+ Fix 2	+ Fix 3	+ Fix 4
A0 Legit use	PASS	PASS	PASS	PASS	PASS
B1 Prompt-only	LEAK	BLOCKED	BLOCKED	BLOCKED	BLOCKED
B2 Spoofed groups	LEAK	LEAK	BLOCKED	BLOCKED	BLOCKED
B3 Lost labels	LEAK	LEAK	LEAK	BLOCKED	BLOCKED
B4 Shared cache	LEAK	LEAK	LEAK	LEAK	BLOCKED

A0 guards against over-blocking: Ben must keep access.

Part D – Prove it (10 min)

1 · Retry your best injection. The model may obey – nothing restricted is in its context.

2 · Run with real answers (slow); also checks for CANARY markers.

3 · Detect it. Find requests that claimed groups the user doesn't have.

```
python3 -m ragkit.attack --llm

grep '"claimed_groups": \[' \
  data/audit.jsonl
```

Debrief and stretch challenges

Debrief questions

1. Why is “it refused when I tested it” not evidence of a secure system?
2. B3 leaked with the filter on. Where else can access-control bugs live?
3. A doc is reclassified after its answers were cached. What still leaks?
4. Which flaw would you most expect in a RAG pilot here, and why?
5. Data-residency rules: which layer enforces them, and how is it like Fix 1?

Stretch challenges (experts)

Reclassification: fix the stale cache after a label change

Fail closed: unlabelled chunks denied to everyone

Write a detection: claimed_groups $\not\subset$ idp_groups

Poisoned document: what can injected text still do?

Port to Chroma: Fix 1 as a metadata where filter

Data residency: filter on a data_region field

Key takeaways

Authorise at retrieval, not in the prompt
Forbidden text never reaches the model

Identity comes from the identity provider
Never from the client request

Labels must survive every pipeline step
Loaders, chunkers, re-indexing. Default to deny

Caches, logs and traces are data paths
Scope them like the data they hold

Test with automated checks
And prove legitimate users keep access



Authorise before the context window, not after.

Next: Module 5 – Agents and automation